

居丽叶的简历项目5：K12数学题解答多模态模型全链路训练



本项目应用场景为数学，可以包装为在豆包爱学、学而思学练机、MOOC等教育应用部门的实习，或者偏多模态基座部门的实习。也可以结合自身经历，将数学场景替换到别的场景。

如果改造为小公司实习，可以把32B模型替换为8B模型，32卡训练改为1机8卡训练

代码解读和代码运行文档：[📖 多模态项目代码解读](#)

自我介绍

面试官您好，我叫xxx，本科毕业于XX大学，硕士就读于XX大学XX学院。在校期间我曾多次获得过奖学金，我的研究方向是多模态大模型和强化学习。

此外，我还有一段实习经历，是在xx公司担任大模型算法工程师/实习生，主要参与的项目是K12数学题解答的多模态模型训练项目。这个项目的背景是增强模型对数学题的解答能力，我参与了整个训练流程的设计和实现，包括CPT、SFT和RL三个阶段的数据构建、训练策略优化等多项工作，积累了大规模多模态模型训练的丰富经验。

以上是我的自我介绍，需要我给您详细介绍一下这个多模态模型训练项目吗？

简历写法

K12数学题解答多模态模型全链路训练 | [公司名称] 多模型算法工程师/实习生 2025.04 – 2025.09

项目背景

OCR+纯文本大模型方案存在识别准确率低、几何图形信息丢失等问题，图片数学解题准确率仅为58%，难以满足业务需求。

解决方案

- **CPT训练**：基于Qwen2.5-VL-32B基座模型，使用Megatron-LM在32张A800 GPU上进行2千万图文对+100B文本数据混合训练；
- **SFT微调**：构建thinking/no-thinking混合训练集（77万条，3:1配比），增强模型指令遵循、视觉特征理解和推理能力，并要求生成指定格式的回复；

- **RL阶段：**使用verl框架进行GSPO训练，经过去重、题型分类、可解答性筛选、Pass@K正确率筛选得到8万条高质量RL训练集，优化奖励函数判题规则，通过异步奖励计算和vLLM推理加速优化训练效率。

项目成果

- **模型性能：**清晰图准确率从68%提升至87%（+19%），模糊图准确率从55%提升至75%（+20%）；纯文本准确率从68%提升至80%（+12%）；
- **Qwen2.5-VL-32B高质量数学训练数据以及高准确率判题规则：**积累了8万条高质量RL阶段训练数据，开发出一套准确率98.5%的奖励函数数学判题规则。

项目描述

项目背景是增强模型对K12中数学题的解答能力。在多模态模型还没火时，最初的方案是用训练好的纯文本大模型进行解题，当用户上传图片题目时，首先使用OCR技术（如PaddleOCR）将图片中的文字和公式识别并转换为文本，然后将文本输入给大模型进行解题。这会面临很多问题，包括但不限于OCR识别准确率低、几何图形信息丢失、额外的时延和成本等。此阶段在清晰图评测集上，图片数学解题准确率仅为58%，主要受限于OCR识别错误和信息丢失。

为了解决OCR方案的局限性，我们决定训练自己的多模态模型，专门针对数学题目场景进行优化。在基座模型选择上，我们调研了LLaVA系列、InternVL系列、CogVLM系列、Qwen-VL系列等多个主流多模态模型。在内部的清晰图评测集和文本评测集上，Qwen2.5-VL系列指标最高，且训练框架成熟，支持Megatron-LM分布式训练，最终选择Qwen2.5-VL-32B作为基座模型。CPT阶段的目标是让模型学会理解数学图像和文本的对应关系，提升数学视觉推理能力。我们收集了约2kw图文对的数学题目数据，包含手写数学题照片、印刷体数学题题目截图、几何图形、函数图像、统计图表等多种类型，数据来源包括内部题库解析、Seed/Gemini等模型蒸馏的合成数据。同时使用100B纯文本数据进行混合训练，以提升模型的通用能力。在此期间做了很多训练策略的优化，比如分阶段训练，序列打包等。经过CPT训练后，模型在多个评测集上取得了显著提升。比如在清晰图评测集上，准确率从基座模型的68%提升至73%（+5%）；在模糊图评测集上，准确率从基座模型的55%提升至60%（+5%）。

接下来SFT阶段，目的是通过高质量指令数据微调，提升模型在数学解题任务上的表现。我们构建了多来源、多模态的SFT训练数据集，采用thinking和no-thinking数据混合训练的策略。thinking部分包括随机采样的30万条视觉数据和27万条纯文本数据（用seed-1.6刷回复），共计约57万条thinking数据。no-thinking部分包括视觉数据2.5万条和文本数据12万条，其中视觉数据重复三次以平衡数据分布（重复后视觉数据变为7.5万条，与文本数据12万条的比例约为1:1.6，避免视觉数据在训练中被文本数据淹没）。数据来源包括从多个内部数据源进行均匀随机采样的解题数据，以及从通用数据中采样的图文交错、OCR、日常场景标题、几何形状识别、计数、统计图、示意图理解等类型的数据。把视觉think/no-think数据混合训练模型对比仅no-think数据训练的模型有5%的提升，并进一步通过resize优化图像预处理，对低质量手写图像识别有显著效果。经过SFT阶段的训练和优化，模型在多个

评测集上取得了显著提升。清晰图评测集准确率从CPT阶段的73%提升至84%（+11%），模糊图评测集准确率从CPT阶段的60%提升至72%（+12%）。

接下来RL阶段，通过强化学习进一步提升模型在数学解题任务上的准确率，特别是在复杂推理任务上的表现。我们构建了多来源、高质量的数学RL训练数据集，原始数学数据共50万条。经过去重、题型分类、答案提取和质量筛选等处理步骤，最终用于训练的数据量约8万条。基于SFT模型，使用GSPO算法进行强化学习训练。使用megatron-lm框架进行分布式训练，在多机A800上训练（32张卡），训练5个epoch。奖励函数使用verl的数学判题规则进行答案验证。经过RL阶段的训练和优化，模型在多个评测集上取得了提升，比如清晰图评测集准确率从SFT阶段的84%提升至87%，模糊图评测集准确率从SFT阶段的72%提升至75%（+3%），训练过程中，最优checkpoint通常在step 400步左右，这些提升主要来自于高质量的RL训练数据构建、奖励函数的优化，以及GSPO算法的有效训练。

后来Qwen3-VL推出后，也在做模型基座的迁移，以及训练MoE架构的模型，在保持性能的前提下降低推理成本，目前正在开展中。

项目相关的问题

1. CPT

1.1 CPT阶段训练设置

- 使用Megatron-LM分布式训练框架，支持多机多卡训练
- 全参数微调（不冻结LLM和视觉编码器），学习率 $5e-6$ ，warmup_iters=1000，最小学习率 $5e-7$ ，使用cosine decay策略
- 全局batch size 512，序列长度8192，使用FP16混合精度训练
- 采用sequence parallel、flash attention等优化技术
- 并行策略：TP4（模型参数在4张GPU间切分，组内使用4路Sequence Parallel切分序列维度），PP4（模型层在4张GPU间切分），DP2（数据在2个GPU组间切分），总共32张A800GPU（ $4 \times 4 \times 2 = 32$ ），训练1个epoch，时间约3周
- 训练速度：FP16精度下达到约2300 tokens/s/gpu

1.2 为什么选择Qwen2.5-VL-32B作为基座模型？

在CPT阶段开始前，我们对多个主流多模态模型进行了调研和对比，包括LLaVA系列（LLaVA-OneVision）、InternVL系列（Intern VL 3）、CogVLM系列（CogVLM2）、Qwen2.5-VL系列等。最终选择Qwen2.5-VL-32B作为基座模型，主要基于以下几个方面的考虑。

首先是性能表现最优。在内部评测中，Qwen2.5-VL-32B在数学题目理解任务上表现最好。在清晰图评测集上，基座模型准确率达到68%，相比LLaVA-OneVision、Intern VL 3、CogVLM2等其他候选模型提升了3-5个百分点。在纯文本和模糊图评测集上，也都取得了最高的准确率，明显优于其他多模态模型。

其次是训练框架成熟。Qwen2.5-VL系列原生支持Megatron-LM分布式训练框架，支持Tensor并行、Pipeline并行、Sequence并行等多种并行方式，能够灵活应对32B规模模型的训练需求。在实际训练中，使用FP16精度训练时，训练速度达到约2300 tokens/s/gpu，训练效率稳定可靠，同时支持8192 tokens的长序列训练。

最后是训练稳定性好。通过合理的学习率设置和warmup策略，训练loss曲线平滑下降。相比之下，我们在测试其他模型时发现，LLaVA和InternVL系列在训练初期容易出现loss波动。基于Qwen2.5-VL-32B基座模型，我们成功完成了CPT、SFT、RL等多个阶段的训练，每个阶段都取得了显著的性能提升，说明该模型具有良好的可扩展性，而其他候选模型在多阶段训练中的稳定性相对较差。

综合以上因素，Qwen2.5-VL-32B在性能、训练框架、架构设计和稳定性等方面相比其他多模态模型都表现出明显优势，是最适合我们数学题目理解任务的基座模型选择。

1.3 CPT阶段的数据是如何构建的？

CPT阶段的数据构建主要包括两个部分：图文对数据和文本数据。

图文对数据：我们收集了约2kw图文对的数学题目数据，包含多种类型：

- 手写数学题照片：学生手写的数学题目照片，包含各种字迹和书写风格
- 印刷体数学题题目截图：从教材、练习册等来源的印刷体数学题目

其中还可能包含子图，这些子图对解题很重要，而OCR方案对子题解析不好，子图类型包括

- 几何图形：包含各种几何图形的题目，如三角形、圆形、多边形等
- 函数图像：包含函数图像的题目，如一次函数、二次函数、三角函数等
- 统计图表：包含统计图表的题目，如柱状图、折线图、饼图等

数据来源包括内部题库、Seed/Gemini等模型蒸馏的合成数据，确保数据的多样性和质量。

文本数据：同时使用100B纯文本数据进行混合训练，以提升模型的通用能力。这些文本数据主要来自通用语料库，帮助模型保持语言理解和生成能力。

通过图文对数据和文本数据的混合训练，模型既能学习到数学图像和文本的对应关系，又能保持通用语言能力，为后续的SFT和RL阶段打下良好基础。

1.4 为什么使用图文对和文本数据混合训练？

在CPT阶段，我们采用图文对数据和文本数据混合训练的策略，主要基于以下几个考虑。

首先是保持模型的通用能力和平衡不同模态。如果只使用图文对数据，模型可能会过度专注于视觉理解，而忽略语言理解和生成能力，也就是模态之间不平衡，会导致模型在纯文本任务上的表现下降。

通过混合100B文本数据，可以确保模型在提升视觉理解能力的同时，保持原有的语言能力，避免能力退化。混合训练能够有效平衡不同模态的能力，避免单一模态的过度优化，这是多模态模型训练中被广泛采用的最佳实践。

其次是提升训练效率和计算资源利用。纯文本数据的训练速度通常比图文对数据更快，因为不需要处理图像编码和视觉特征提取。图像编码需要额外的计算开销，包括图像预处理、视觉编码器前向传播等步骤。混合训练可以在保证视觉能力提升的同时，提高整体训练效率，更好地利用计算资源。在实际训练中，我们观察到混合训练的整体吞吐量比纯图文对训练提升了约15-20%。

第三是提升模型泛化能力和鲁棒性。数学题目场景中，有些题目是纯文本的，有些题目包含图像。如果只使用图文对数据，模型可能对纯文本题目的处理能力不足，也可能会过度拟合特定的图像-文本对应关系，导致在未见过的图像类型上表现不佳。混合训练可以让模型更好地适应不同的输入格式，同时处理纯文本和图文混合两种输入，这种多样性有助于提升模型的泛化能力和鲁棒性。文本数据通常比图文对数据更容易获取，数据量更大，能够提供更丰富的语言模式和知识。在实际训练中，我们发现混合训练的模型在纯文本评测集上的表现比纯图文对训练的模型提升了约1-2个百分点。

实验结果表明，混合训练策略是有效的，模型在CPT阶段在多个评测集上都取得了显著提升。清晰图评测集准确率从基座模型的68%提升至73%，纯文本评测集准确率从68%提升至71%，说明混合训练既提升了视觉理解能力，又保持了文本理解能力。

1.5 FP16混合精度训练的优势是什么？

FP16混合精度训练相比FP32全精度训练，主要有以下优势。

1. 训练速度提升。在FP16精度下，训练速度相比FP32全精度训练有明显提升，达到约2300 tokens/s/gpu。这主要得益于FP16精度下计算量更小，内存带宽占用更低，同时现代GPU对FP16计算有硬件加速支持。
2. 显存占用降低。FP16精度下，模型参数和激活值占用的显存相比FP32减少约50%，可以在相同的硬件条件下支持更大的batch size或更长的序列长度。
3. 训练效果稳定。通过合理的训练策略和loss scaling技术，FP16训练的模型效果与FP32训练相当，在各项评测指标上没有明显下降。

此外，相比于BF16，FP16有以下优势：

1. 更高的数值精度：FP16拥有10位尾数，而BF16只有7位。BF16为了保持与FP32相同的动态范围，牺牲了部分精度。这意味着FP16在表示微小数值变化时更精确，在数学推理这种对逻辑严密性要求极高的任务中，有助于模型捕捉更细粒度的特征差异。
2. 推理部署的无缝衔接：推理框架对fp16的优化更深，使用FP16训练可以确保训练与推理一致。

1.6 如何解决训练中的OOM问题？

在使用FP16精度训练时，某些step可能会出现OOM（显存溢出）问题。我们采用了以下优化措施：

1. **启用Sequence Parallel（序列并行）**：在 Megatron-LM 框架下，针对 8192 长序列开启序列并行。在 TP=4 的各组内，将序列维度切分至不同 GPU 处处理（单卡仅需处理 2048 Tokens）。通过将原本在 TP 组内冗余存储的非线性层激活值（如 LayerNorm、Dropout、Residual Connection）进行分布式存储，将这部分显存开销降低至原来的 1/4。底层实现上，将原有的 All-Reduce 通信重构为 Reduce-Scatter 与 All-Gather 的组合，在保持通信总量不变的前提下，消除了单卡显存瓶颈，使得 32B 模型在有限显存下能支持更大的 Batch Size 或更长的序列窗口，且不改变总体集群拓扑（TP x PP x DP = 32）。
2. **使用Gradient Checkpointing（梯度检查点）**：这是一种以时间换空间的优化技术。在标准的前向传播过程中，需要保存所有中间激活值用于反向传播计算梯度，这会占用大量显存。Gradient Checkpointing 的策略是只保存部分关键激活值，在反向传播时重新计算那些被丢弃的激活值。虽然这会增加计算时间，但可以将激活值显存大幅降低。
3. **调整并行策略**：优化 micro_batch_size 和 pipeline 并行策略，平衡显存占用和训练效率。Pipeline 并行将模型的不同层分配到不同的 GPU 上，每个 GPU 只需要保存当前层的参数和激活值，可以大幅降低单卡显存占用。但 Pipeline 并行会引入流水线气泡，降低训练效率。我们通过调整 micro_batch_size 来平衡这个问题，较小的 micro_batch_size 可以减少单卡显存占用，并减少流水线气泡，但会牺牲计算效率。我们最终将 micro_batch_size 设置为 4，这是一个平衡点，既保证了训练不会 OOM，又维持了较高的训练效率。

1.7 如何解决训练初期的loss抖增问题？

在 CPT 训练初期，我确实遇到了 Loss 突然抖增（Loss Spike）的问题。针对这个问题，我主要从数据和超参数两个维度进行了排查和解决。

第一方面是**数据排查与清洗**。面对 CPT 阶段的海量数据，首先通过小规模采样，识别出坏图、乱码等典型脏数据模式；然后过滤掉这些脏数据，移除了约 1.5% 的低质量数据。同时，我还统计了数据的长度分布，设定 6000 为截断阈值，过滤掉可能导致显存溢出的超长序列。

第二方面是**超参数与策略调整**。首先，我检查了基础超参数。确认了 Weight Decay 没有设置过大，避免模型参数被强行拉向 0 而震荡；同时设置梯度裁剪生效，防止个别极端样本导致梯度爆炸。其次，我重点调整了**学习率策略**。Loss 抖增最直接的原因通常是学习率过大或热身不足。我将初始学习率降低到了 $5e-6$ ，并配合 1000 步的 Warmup，让模型有一个适应梯度的过程。最后，为了处理新加入的随机初始化 Embedding，我采用了“**分阶段训练**”策略。先冻结模型主体，只训练 Embedding 层 500 步，让 Loss 平滑下降到 3.0 左右，再解冻主体进行全量训练。

1.8 分阶段训练策略是什么？为什么要这样做？

第一阶段是冻结 LLM 与 Vision Encoder 主干，仅解冻 Projector（连接层）以及扩充后的 Input Embedding / Output LM Head（针对数学场景常用的 Token）。采用较大的学习率 $1e-5$ ，进行约 500 步的预热训练。目的是解决随机初始化带来的“梯度冲击”。由于数学 Token 和投影层参数与模型预训练权重处于完全不同的分布空间，直接全量训练会导致巨大的梯度范数破坏模型原有的语义记

忆。通过这一“软着陆”阶段，让新增参数在不干扰主干的情况下快速收敛至模型主空间，实现 Loss 从 4.1 到 3.0 的跨越，并使梯度范数回归平稳。

第二阶段是全量微调，也就是正常的训练过程。在这个阶段，我解冻了Backbone，进行全参数训练。学习率下调至 $5e-6$ ，进行Warmup和Cosine Decay。此时，由于Embedding层和projector已经“对齐”，模型的主体参数可以专注于学习如何处理新的多模态特征，进一步优化生成能力。最终，Loss 从3.0平滑下降至1.2，整个训练过程非常稳定，没有出现Loss Spike或模型坍塌。

1.9 如何进行数据去重和去污染？

1. 多模态数据去重

- 物理级去重：首先基于题目ID和图片ID进行硬匹配，剔除完全重复的记录

2. 数据去污染

为了防止模型在评测集中表现出数据泄露，我针对 6 大核心评测集 实施了闭环过滤：

- N-gram 检索：采用 8-gram 匹配算法。一旦训练集中的连续 8 个 Token 与评测集重合，该样本即被判定为污染数据并予以剔除。

1.10 序列打包技术是怎么做的？

序列打包技术通过将多个短序列合并到一个固定长度的序列中，减少padding浪费。不同样本的序列长度差异很大，如果每个样本都padding到最大长度比如8192，会产生大量无效的padding token。打包策略是将多个短序列按顺序拼接接到8192 tokens的序列中，使用attention mask标记每个样本的边界，确保不同样本之间不会相互影响。当当前序列长度不足时，继续添加下一个样本，如果加上下一个样本会超过8192，就不加这个样本，直接用padding填充剩余部分。举个例子，样本1是2000 tokens，样本2是5000 tokens，样本3是800 tokens，这三个加起来是7800 tokens，如果下一个样本是500 tokens，加上后超出8192，就不再继续添加。

序列打包的优势是减少padding浪费，有效token比例大幅提升，能提高训练速度，减少无效计算。训练过程中要确保不同样本之间不相互影响，只对有效token计算损失，并且padding部分不参与损失计算。

1.11 如何平衡图文数据和纯文本数据的配比

1. Token级的总量控制：我们构建了 100B 文本 搭配 2kw 图文对 的数据集。从Token量级来看，文本数据占据绝对主导（约 90%）。这种配比的核心逻辑是：CPT 的核心任务是保持 LLM 强大的推理基座，同时将视觉模态对齐到这个基座上。大量的文本数据充当了“正则化”的角色，防止模型因为过拟合视觉特征而丢失通用的语言逻辑能力。

2. 动态退火采样：在训练的最后阶段（Decay阶段，训练了2000步），我们实施了**“高质量数据退火”。我们从全量数据中筛选出约 10% 的核心高质量数据**（主要包含 seed-1.6 蒸馏的数学 CoT 文本、人工筛选的高分辨率图文题解），并将这部分数据的采样权重提升 3-5 倍。这种策略让模型在参数收敛的最后阶段，集中学习最高质量的知识分布，有效提升了最终的解题准确率。

1.12 CPT阶段的训练数据格式

代码块

```
1  图文对数据示例
2  {
3      "image_file_path": "/math/1.png",
4      "text": "求解方程： $2x + 3 = 7$ \n\n解题步骤：\n1. 将常数项3移到等式右边： $2x = 7 - 3$ \n2. 计算右边： $2x = 4$ \n3. 两边同时除以2： $x = 2$ \n\n因此，方程的解是 $x = 2$ 。",
5      "subject": "math",
6      "grade": 3
7  }
8
9
10  纯文本数据示例
11  {
12      "text": "在数学中，二次方程是形如 $ax^2 + bx + c = 0$ 的方程，其中a、b、c是常数，且 $a \neq 0$ 。解二次方程的方法有因式分解法、配方法、公式法等。公式法是最通用的方法，其解为 $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 。判别式 $\Delta = b^2 - 4ac$ 决定了方程的解的性质：当 $\Delta > 0$ 时，方程有两个不同的实数解；当 $\Delta = 0$ 时，方程有两个相同的实数解；当 $\Delta < 0$ 时，方程没有实数解，但有两个复数解。",
13      "subject": "math",
14      "grade": 3
15  }
```

1.13 你们的评测体系是怎么构建的？

为了全面评估模型能力，我构建了3个维度的专用评测集，总共六个能力评测：

- **学段维度**：小学、初中、高中三个难度梯度的数学题，重点考察数学推理能力。
- **视觉鲁棒性维度**：清晰图（原题截图）和模糊图（模拟手机拍摄、光线不足场景），重点考察OCR和视觉抗干扰能力。
- **基座维度**：纯文本逻辑题，确保多模态训练没有损害语言模型的根本。

2. SFT

2.1 SFT训练设置

- 基于CPT模型权重，全参数微调
- 学习率 $5e-6 \rightarrow 1e-6$ ，最小学习率 $1e-6$ ，使用cosine decay策略
- 全局batch size 512，序列长度8192，使用Megatron-LM框架
- 并行策略：TP4，PP4，DP2，总共32张A800GPU ($4 \times 4 \times 2 = 32$)
- 训练3个epoch，在32张A800GPU上训练约3天
- 采用损失掩码机制，只对assistant回复部分计算损失，避免模型学习预测用户输入

2.2 SFT阶段，Vision Encoder和LLM是全量微调还是部分冻结？为什么？

在我们的SFT阶段，我们采用了全量微调（Full Fine-tuning）的策略，即Vision Encoder和LLM的所有参数都参与更新。

1. 任务目标的升维（从描述到推理）：CPT阶段的任务本质是看图说话，视觉编码器学到的是“这是一张三角形的图”这样的全局语义特征。而SFT阶段的任务是复杂的数学推理（CoT），模型需要关注的是解题关键线索（如三角形辅助线的虚实、坐标轴的刻度值、角标的细微差别）。冻结Vision Encoder会限制模型提取这些高频细节的能力，导致看得见图，但看不准细节。
2. 视觉与思维链的深度对齐：SFT过程中，LLM通过学习特定的推理模式，其语义空间会向逻辑推导大幅偏移。若视觉编码器保持冻结，其输出的特征仍停留在原始的对齐空间，会导致视觉表征与LLM的推理节奏出现空间错位。全量微调能促使Vision Encoder主动适配LLM的推理逻辑，输出更具逻辑判别性的视觉特征，从而打破模态隔离带来的性能瓶颈。

2.3 SFT训练中为什么采用图文混合训练？

1. 防止文本推理能力对视觉特征的过度依赖：在SFT阶段，如果模型仅接触图文数据，它倾向于建立“看到图片才能推理”的强依赖路径，导致在处理纯文本问题时，由于缺乏视觉输入而无所适从。通过混合纯文本数据，我们强制模型在没有视觉辅助的情况下也能激活其逻辑推理回路，从而解耦了“逻辑推理”与“视觉输入”的强绑定关系，确保模型在有图和无图场景下都能稳定发挥。
2. 跨模态能力迁移：实验发现，强大的文本推理能力可以正向迁移到多模态任务中。通过混合训练，模型学会了将在纯文本上学到的复杂逻辑推演模式，应用到需要结合视觉信息的题目中，实现了“以文促图”。
3. 平衡数据分布：真实应用场景中，用户的问题既包含纯文本也包含图文。混合训练模拟了这种真实分布，让模型能够灵活处理不同模态的输入，而不是过度拟合到某一种特定的输入模式。

2.4 SFT阶段的数据分布是什么？

在SFT阶段，我们构建了多来源、多模态的训练数据集，采用了系统化的数据构建策略。

首先是数据来源。我们只使用了业务数据，没有使用通用场景数据。数据全部来源于内部积累的小学、初中、高中数学题库，这样做是为了让模型完全专注于数学解题能力的提升，避免通用数据分散

模型的注意力。

其次是thinking和no-thinking数据的配比。thinking部分包括约57万条数据，其中30万条是视觉数据，27万条是纯文本数据，这些数据都使用seed-1.6模型生成高质量的推理回复。no-thinking部分包括约20万条数据，其中视觉数据2.5万条（重复三次后变为7.5万条），文本数据12万条。我们特意对视觉数据进行了重复，使得重复后的视觉数据与文本数据的比例约为1:1.6，这样避免了视觉数据在训练中被文本数据淹没，保证了多模态能力的平衡发展。

最后是数据质量控制。对于thinking数据，我们使用seed-1.6模型进行回复生成，确保了推理过程的质量。对于no-thinking数据，我们通过格式统一处理，添加空的`<think></think>`标签，保持与thinking数据格式的一致性。

2.5 CPT和SFT阶段的训练数据格式有什么区别？

- CPT阶段目标是让模型学习数学图像和文本的对应关系，提升数学视觉推理能力。对于图文对数据，会将图像和对应的文本描述直接拼接，模拟模型在预训练阶段看到的自然语言分布。对于纯文本数据，就是简单的文本序列，不包含conversations结构。数据经过预处理后，通过序列打包技术将多个短序列合并到固定长度的序列中。
- SFT阶段使用conversations格式，包含system、user、assistant三个角色。assistant的内容包含 `<think>` 和 `<answer>` 标签，对于no-thinking数据，`<think>` 标签内容为空。

格式差异的原因是，CPT阶段采用扁平化格式，让模型直接学习领域知识。如果强行使用conversations格式，会导致模型在学习领域知识时受到大量重复角色标签（User/Assistant）的干扰，降低信息增量的吸收效率。SFT阶段是指令微调，需要训练模型的指令遵循和对话能力，因此必须使用conversations格式，让模型学习如何根据system prompt和user输入生成符合要求的assistant回复。这种格式差异反映了从基础能力学习到任务能力学习的转变。

2.6 为什么think和no-think数据可以一起训练？

首先，对于所有no-thinking数据，我们均按照qwen2.5格式添加空<think></think>标签，格式为<think>\n\n</think>\n\n{content}，以保持与thinking数据格式的一致性。这样两种数据在格式上保持一致，可以混合训练。

其次，模型具备条件生成能力，训练机制支持混合学习。现代大语言模型本质上是一个条件生成模型，能够根据输入格式自动选择生成模式。在训练过程中，模型通过损失函数学习预测下一个token，无论是thinking数据还是no-thinking数据，模型都在学习正确的token序列。thinking数据包含完整的推理过程，模型学习如何生成详细的推理步骤；no-thinking数据虽然推理过程被隐藏，但模型仍然学习如何直接生成答案。这两种学习目标并不冲突，反而能够相互促进，thinking数据中的推理过程能够帮助模型理解问题的本质，这种理解能力也会体现在no-thinking数据的生成质量上。这种设计类似于多任务学习，让模型同时掌握"详细推理"和"直接回答"两种能力。

第三，混合训练能够提升模型的综合能力和鲁棒性，实验验证了其有效性。thinking数据帮助模型学习复杂问题的推理过程，提升模型在困难问题上的表现；no-thinking数据帮助模型学习简洁高效的直接回答，提升模型在简单问题上的响应速度。混合训练使得模型能够根据问题难度自动选择是否展示推理过程，这种灵活性在实际应用中非常重要。在数据迭代过程中，我们发现把视觉think/no-think数据混合训练模型对比仅no-think数据训练的模型有5%的提升，这证明了混合训练策略的有效性。混合训练的模型在复杂推理任务上表现更好，同时在简单任务上也能保持高效，这种提升是单一类型数据训练无法达到的。

2.7 SFT阶段如何优化图像预处理？

在 SFT 阶段，针对手写公式和几何图形识别率低的问题，优化核心在于提高最小像素限制min_pixels为224，过低的配置会导致小尺寸图片（或长条状图片）被过度压缩。通过调高 min_pixels，可以强制模型对图像分配更多的视觉 Token。这种做法显著提升了视觉采样密度，使得手写笔画、几何线条等高频细节在经过视觉编码器时，能够保留更完整的空间拓扑特征。

结合 SFT 阶段的针对性训练，模型不仅学习到了更丰富的视觉特征，还通过指令微调增强了对低质量输入的容错能力。实验数据显示，通过提升最小采样分辨率，模型在手写数学公式和几何识别任务上的准确率从 60% 提升至 72%。这一改进在不显著增加计算负担的情况下，极大增强了模型在复杂 OCR 场景下的鲁棒性。

2.8 SFT阶段如何平衡thinking和no-thinking数据的比例？

在SFT阶段，我们采用了thinking和no-thinking数据混合训练的策略，数据比例的平衡是关键。我们的配比策略是**thinking数据为主，no-thinking数据为辅**，整体比例约为3:1。

这个比例的选择并非随机，而是基于对帕累托平衡点的探索。**Thinking数据（57万条）**作为主力，其包含的CoT推理链能有效对齐视觉特征与逻辑步骤。参考DeepSeek-R1与Qwen2.5-VL的实践，复杂的视觉数学和几何问题必须依赖密集的推理轨迹来抑制幻觉，3:1的高占比确保了模型在处理复杂任务时能优先触发逻辑推理。**No-thinking数据（20万条）**作为补充，其存在的意义是防止模型产生"冗余回复"。它能训练模型在面对简单指令时保持高效、简洁，避免过度推理导致的响应延迟。

为了防止视觉特征在多模态SFT中被文本淹没，我们特别采用了**过采样策略**。no-thinking的视觉数据原始仅2.5万条，我们对其进行了3倍过采样提升至7.5万条，使得视觉数据与文本数据的最终比例达到约1:1.6。这一比例确保了视觉编码器和LLM Backbone在梯度更新时保持均衡，防止模型退化为纯文本模型。

实验结果显示，这种3:1的混合分布相比于传统的纯no-thinking训练，在多个多模态评测集上取得了5%的绝对增量。

2.9 SFT训练中，图像是如何转换为Token输入到LLM的？

首先是视觉编码Vision Encoding。输入的原始图像，比如分辨率为224x224，首先会被输入到视觉编码器Vision Encoder，通常是ViT结构。Vision Encoder会将图像切分成Patch，比如14x14大小，并提取出高维的视觉特征Feature Map。

接着是维度映射Projection。提取出的视觉特征会经过一个Projector适配器，通常是MLP结构。它的作用是将视觉特征的维度映射到LLM的Embedding空间维度，使其能被LLM理解。同时，这里还会进行Token数量的压缩。

最后是Embedding替换Embedding Replacement。在构建SFT训练数据时，我们会在文本的Input IDs中插入特殊的占位符。在数据进入LLM的Embedding层之前，会找到这些占位符的Embedding，并将其替换为前一步生成的Visual Tokens Embedding。

2.10 SFT阶段数据格式

代码块

```
1
2  {
3    "conversations": [
4      {
5        "role": "system",
6        "content": "你是一个数学解题助手，能够帮助用户解决数学问题。"
7      },
8      {
9        "role": "user",
10       "content": [
11         {
12           "type": "text",
13           "text": "求解方程： $2x + 3 = 7$ "
14         },
15         {
16           "type": "image_file_path",
17           "image_file_path": "/math/173a42bd2eb871a.png"
18         }
19       ]
20     },
21     {
22       "role": "assistant",
23       "content": "<think>\n我需要解这个方程： $2x + 3 = 7$ \n首先，我将3移到右边： $2x = 7 - 3$ \n计算： $2x = 4$ \n然后，两边同时除以2： $x = 2$ \n</think>\n<answer> $x = 2$ </answer>"
24     }
25   ],
26   "question_id": "111",
```

```
27     "subject": "math",
28     "grade": 3,
29     "answer": "$x = 2$"
30 }
31
32 对于纯文本数据，user的content数组中不包含`image_file_path`类型的元素；对于no-thinking
    数据，`<think>`标签内容为空。
33
```

2.11 SFT模型的输出格式怎么约定？

为了确保RL阶段奖励计算的准确性，SFT模型的输出（特别是`<answer>`标签内的答案部分）遵循一套严格的标准化格式约定。因此在SFT数据构造阶段，我们就对训练数据的label进行严格的清洗和格式化，使其符合以下规范，从而让模型学会生成标准化的输出。这些约定主要涵盖了数学领域的多种情况：

1. 基础LaTeX规范：

- 所有的数学公式和答案需用 `$` 包裹，例如 `$x=2$`。
- 严禁使用 `\boxed{}` 包裹答案。
- 严禁自行添加解释性文字（如答案是、等于），只保留纯数值或表达式。

2. 题型标准化：

- 判断题：统一转化为 `\checkmark` 和 `\times`。
- 选择题：仅输出选项字母（如 `$A$`），多选时用逗号分隔（如 `$A,B$`）。
- 填空/解答题：
 - 数值：保留最简形式，如 `$1/2$`（分数）或 `$0.5$`（小数）。
 - 表达式：必须规范化，例如 `$y=3x+1$`。
 - 区间/集合/坐标：视为一个整体，如 `$(1, 2)$`、`$[0, +\infty)$`。

3. 多答案处理：

- 独立小题：如果一个大题包含多个独立的小题，在`<answer>`标签内用`<split>`分割每个小题的答案。
- 多解情况：对于同一个填空或解答，如果存在多个正确解（如方程的两个根），使用`<or>`进行分割。

4. 特殊符号规范：

- 百分数：使用 `\%` 转义，如 `$50\%$`。
- 度数：使用 `^{\circ}`，如 `$30^{\circ}$`。
- 文本描述：如果答案必须包含中文或英文描述，使用 `\text{}` 包裹，如 `$_{\text{无关}}$`。

通过在SFT阶段就强制模型遵循这套格式，RL阶段的奖励函数可以直接解析 `<answer>` 标签内容，极大简化了后处理流程并提高了判题的准确性。

2.12 SFT/RL数据如何构造符合规范的标准化答案？

为了让SFT模型的输出满足严格的格式要求，我们采用了一套基于大模型的自动化改写与验证流程。具体步骤包括：首先将SFT模型输出规范详细编写进Prompt中；然后调用大模型将原始数据的回复改写为符合规范的格式；接着通过构建包含1000条数据的Golden评测集进行人工校验，并根据Bad Case不断迭代优化Prompt；最终，经过多轮迭代，改写后的回复完全满足格式要求的比率高达98.5%。我们对所有SFT数据的答案进行了批量改写，从而构建了高质量的标准化SFT数据集。

2.13 SFT/RL数据的标准答案是怎么来的？

SFT数据的标准答案主要来源于两个渠道：首先是利用现有的高质量题库，直接提取其自带的正确答案作为基础；其次，对于部分缺失答案或原答案解析不清晰的题目，我们利用 Seed / Gemini等强模型进行推理生成。获取到这些初始答案后，再进行统一的批量改写成指定的格式，方便RL阶段进行训练。

3. RL

3.1 强化学习阶段训练设置

- **训练框架**：使用verl框架进行强化学习训练，底层使用Megatron-LM进行模型训练
- **模型配置**：基于SFT模型，使用GSPO算法进行全量参数训练
- **训练参数**：全局batch size 512，序列长度8192，rollout_n=16（每条Query生成16条Rollout回复），ppo_mini_batch=128，学习率1e-6。
- **并行策略**：TP4，PP4，DP2
- **训练配置**：训练5个epoch，每个epoch步数 = 数据量 / Global Batch Size = 80000 / 512 $\approx$ 156步，总计约780步
- **推理配置**：使用vLLM进行推理，支持PagedAttention和Continuous Batching，提升rollout生成效率
- **奖励计算**：使用异步reward计算以提升训练速度，通过verl的数学判题规则进行答案验证，支持批量计算和异步处理
- **训练时间**：每个step需要约20分钟（包括生成8192个rollout、异步计算reward、反向传播等），总训练时间约8-10天

3.2 奖励函数具体是怎么设置的？

在强化学习训练中，我们的奖励函数采用的是**基于规则的确定性验证**。得益于我们在SFT阶段强制实施的严格输出格式规范，判题逻辑变得非常高效且准确。

具体实现流程如下：

1. 答案提取：由于模型输出被严格约束在 `<answer>` 标签内，直接使用正则表达式提取 `<answer>` 和 `</answer>` 之间的内容。这种方式极大地简化了答案抽取流程，避免了因格式混乱（如 `boxed` 标签）导致的误判，同时也保证了思维链与最终答案的清晰分离。
2. 答案标准化与比对：利用 verl 框架官方提供的数学验证库对提取出的预测答案和标准答案（Ground Truth）进行等价性比对。Verl内置的数学验证器能够智能识别数值和表达式的等价性，例如它可以准确判断 `$x=1/2$` 与 `$x=0.5$` 为同一数值，或 `$y=x+1$` 与 `$y=1+x$` 为等价表达式。对于包含多个小题的情况，利用 `<split>` 符号对答案进行分割。通过这种逐项比对，只有当所有小题的答案都完全正确时，给予 `Reward = 1.0`；任何一处错误或格式违规，给予 `Reward = 0.0`。

通过这种“SFT格式约束 + Verl数学验证”的组合策略，我们极大消除了因格式解析错误导致的误判。在包含各类题型（选择、填空、解答）的内部评测集上，这套规则判题系统与人工专家判题的**一致率高达 98.5%**，为RL训练提供了极其稳定和可信的监督信号。

3.3 RL数据具体是怎么处理的？

RL训练数据构建是一个多步骤的数据处理流程。首先，为了保证数据的多样性和难度上限，我们构建了一个包含 50万条原始数学题目的基础池。这50万条数据全部来自内部积累的高质量K12真题库（涵盖小初高全学段），其中图文对数据和纯文本数据的比例约为1:1（各约25万条）。从这50万条原始数据出发，我们经过层层筛选得到最终的训练集。具体流程如下：

步骤一：去重处理

- 处理方法：基于题目id和图片id进行去重，去除重复的题目数据
- 处理后数据量：30万条（去除20万条重复数据）

步骤二：数学题型分类

- 处理方法：将题目按照题型进行分类，分为选择题、判断题、填空题、解答题四类。使用大模型对题目进行自动分类
- 处理后数据量：18万条（筛选出符合四类题型的题目，去除其他类型(比如证明题)或无法分类的题目)

步骤三：标准答案提取与可解答性筛选

- 处理方法：首先使用多个外部模型（doubao、GPT、Gemini等）对题目进行解答，从外部模型回复中提取答案，同时从题库中提取标准答案。然后使用SFT阶段答案改写的prompt调用seed-1.6模型将提取出的答案改写为符合判题规则要求的格式：对于选择题，提取选项字母（如A、B、C、D）；对于判断题，提取对错标记（如`\checkmark`或`\times`）；对于填空题和解答题，提取数值或表达式答案。改写完成后，进行可解答性筛选，利用判题规则验证改写后的答案是否正确，只保留至少有一个模型能够做对的题目，过滤掉残题、模糊题、标注错误等低质量数据。
- 处理后数据量：12万条（保留所有四类题型，只过滤掉无法提取答案、无法改写为符合要求的格式或无法被模型正确解答的题目）

步骤四：正确率筛选

- 处理方法：基于SFT模型多次尝试后的正确率，筛选出适合RL训练的题目。对每个题目使用SFT模型进行pass@6采样，筛选出有对有错的题目（即SFT模型在多次尝试中既有做对也有做错的题目）。这样在RL训练时，对于这些题目，模型会生成16个rollout，由于题目本身在SFT模型上就表现出有对有错的特点，RL模型生成的rollout之间更容易产生reward差异，从而在组内产生足够的reward方差，可以计算组内相对优势（advantage）并产生梯度进行训练。如果选择全对或全错的题目（全错的题目也可以加入训练，随着训练进行，模型能力增强可能会变成有对有错），组内rollout的reward会过于一致，导致组内reward方差过小，无法产生有效的梯度信号
- 处理后数据量：8万条（筛选出适合RL训练的题目，具有明确的题型分类、标准答案和适合RL训练的难度分布）

通过以上四个步骤的数据处理流程，我们从50万条原始数据逐步筛选和优化，最终得到8万条高质量的RL训练数据。

3.4 为什么用GSPO不是GRPO?

GSPO是在GRPO基础上的改进算法，两者都抛弃了PPO中的Critic模型，采用组内相对优势来更新策略。但它们在实现细节上存在关键差异：

GSPO的优势：首先，GSPO采用序列级别的优化，定义重要性比率基于序列似然，执行序列级别的裁剪、奖励和优化，而GRPO采用token级别的重要性比率。序列级别的处理使得GSPO能够更好地处理长文本生成任务，避免了token级别优化可能带来的不稳定性。其次，相比GRPO，GSPO在训练效率和最终性能上都表现更优，能够更快地收敛到更好的策略。第三，GSPO特别稳定了MoE的RL训练，这对于大规模模型的训练至关重要。

3.5 RL阶段Vision Encoder和LLM的冻结策略是怎样的？

在RL阶段，我们采取的策略是：**冻结Vision Encoder，只微调LLM的主体参数**。这样做的原因主要有以下几点：

1. **资源优化与训练吞吐量**：RL训练对计算资源消耗极大，且极其依赖大规模采样。冻结Vision Encoder可以显著降低显存占用和反向传播的计算量。这部分节省下来的资源至关重要，它允许我们将Rollout数量设得更大，从而获得更低方差的梯度估计，稳定训练过程。
2. **防止视觉表征退化**：RL阶段的奖励信号通常是基于最终答案正确性的标量信号，只有对/错。这种针对逻辑推理的文本反馈信号，并不包含关于图像内容的细粒度监督信息。如果强行用这种信号去更新底层的Vision Encoder，不仅难以提升视觉感知能力，反而极易引入噪声，导致Vision Encoder为了“迎合”Reward而学习到错误的shortcut（如过度关注特定的OCR噪声），破坏在CPT/SFT阶段学到的通用视觉特征。

3.6 如何防止Reward Hacking?

1. **严谨的判题规则**：我们拒绝使用简单的字符串匹配，而是直接采用了verl框架官方提供的数学验证库。该验证库支持数值、表达式、LaTeX等多种格式的等价性验证，能够准确判断模型输出答案的正确性，防止模型通过输出奇怪格式或利用规则漏洞“蒙混过关”。
2. **强制思维链（Chain of Thought）与格式约束**：为了防止模型（特别是处理选择题时）跳过推理过程直接猜测答案，我们强制要求模型在输出最终答案标签`<answer>`之前，必须先输出`<think>`标签。奖励函数不仅检查最终答案的正确性，还会检查`<think>`标签的存在性和完整性。如果模型直接输出答案而没有推理过程，或者格式不规范，直接给予惩罚（Reward=0），迫使模型遵循规范的思考-回答模式。
3. **动态难度筛选（Pass@K机制）**：通过前置的Pass@K（如Pass@6）数据筛选，我们剔除掉了那些模型单纯靠猜测就能高概率做对的简单题，以及完全无法理解的错误题，只保留处于模型“能力边界”上、既有做对也有做错记录的题目。这种筛选机制确保了模型无法通过简单的统计规律或猜测在训练集中获得稳定的高分，必须通过优化推理路径来提升解决边缘难题的成功率。

3.7 混合模态RL训练中，文本数据的作用是什么？

在多模态模型RL训练中，我们不仅使用了图文对数据，还混合了大量的**纯文本数学推理数据**。这么做的主要原因有两点：

1. **适配真实场景与解耦模态依赖**：真实应用场景中，用户的问题既包含纯文本也包含图文。混合训练模拟了这种真实分布。同时，如果仅使用图文数据，模型容易形成对视觉特征的过度依赖（如看到图才推理）。加入纯文本数据，强制模型在没有视觉辅助的情况下也能激活逻辑推理回路，从而解耦了逻辑推理与视觉输入的强绑定，确保模型在优化视觉任务的同时，不遗忘底层的**数学推理能力**。
2. **跨模态增益**：我们的实验结果显示，混合训练带来了显著的全方位提升。对比SFT阶段和RL阶段（32B模型）的评测指标：
 - 文本解题能力提升：文本评测集的准确率从78%提升至80%（+2%）。
 - 视觉解题能力提升：清晰图评测集准确率从84%提升至87%（+3%），模糊图评测集准确率从72%提升至75%（+3%）。

这表明，强大的数学文本推理能力可以正向迁移到视觉任务中，帮助模型更好地理解和推理复杂的几何或图表问题，实现了以文促图的协同效应。

3.8 RL训练中做了哪些优化来加速训练？

在RL训练过程中，我们遇到了训练速度慢的问题，严重影响了训练效率。为了解决这个问题，我们从多个维度进行了优化：

1. CPU-GPU计算流水线并行：在标准的RL训练中，流程通常是串行的：`生成回复(GPU) -> 判题(CPU) -> 计算 LogProb(GPU) -> 更新`。由于数学判题逻辑复杂（涉及verl的数学判题规则的解析与验证），CPU计算耗时较长，导致GPU在判题期间处于闲置状态。我们优化了这一流程，将CPU判题和GPU计算LogProb并行化。当GPU在计算LogProb时，CPU同时在后台进行判题。由于判题时间通常小于或接近LogProb计算时间，这种并行策略成功“掩盖”了判题的延迟，显著提升了整体训练吞吐量。
2. 推理加速优化：使用vLLM进行rollout生成，支持PagedAttention和Continuous Batching技术，显著提升推理效率。vLLM的PagedAttention机制可以更高效地管理KV Cache，减少显存碎片，支持更长的序列长度。Continuous Batching可以动态处理不同长度的请求，提高GPU利用率。
3. 批量计算优化：奖励函数支持批量计算，可以同时处理多个rollout的判题任务，提高计算效率。

3.9 RL训练中的可解答性筛选（可解答性验证）是什么？为什么需要这一步？

在RL数据处理的“可解答性筛选”步骤中，我们首先使用多个外部模型对题目进行解答，从模型回复中提取答案，然后使用SFT阶段答案改写的prompt将答案改写为符合内部判题规则要求的格式。改写完成后，利用判题规则对改写后的答案进行验证，我们只保留那些**至少有一个模型能够做对**的题目。这一步的目的是过滤掉低质量的训练数据。原始训练数据并不完全干净，可能存在以下问题：残题（题目本身不完整或存在错误）、模糊题（图片质量差、OCR识别错误导致题目无法正确理解）、标注错误（Ground Truth答案本身是错误的）、歧义题（题目表述不清，存在多种理解方式）。如果将这些低质量数据用于RL训练，会导致奖励信号混乱，严重损害训练效果。通过可解答性筛选，确保只有高质量的、可被正确解答的题目才会进入后续的训练流程。

3.10 RL训练为什么要进行正确率筛选（Pass@K筛选）？

在RL数据处理的最后一步，我们会基于SFT模型进行多次采样（如Pass@6），只保留那些有对有错的题目和全错的题目。这样做的核心原因在于构建有效的梯度信号：

在RL训练（如GRPO/GSPO）中，模型会对同一道题生成多个（如16个）Rollout，并通过计算组内相对优势来更新参数。如果保留全对的简单题（Reward全为1）或全错的难题（Reward全为0），组内优势值均为0，无法产生有效的梯度信号（全错的题目也可以加入训练，随着训练进行，模型能力增强可能会变成有对有错）。只有针对那些模型“跳一跳能够得着”的题目，这组Rollout中才会既有正确答案也有错误答案。RL算法通过对比正负样本的推理差异，强化导致正确的逻辑路径，抑制导致错误的路径，从而实现有效的策略改进。

3.11 RL训练中如何优化verl框架官方提供的数学验证库判题的准确性？

在RL训练初期，我们发现verl框架官方提供的数学验证库在多模态评测集上的判题结果与人工判题的一致率只有53%，这严重影响了奖励信号的准确性。为了提高判题准确性，我们进行了以下改进：

1. 只对填空题、选择题、解答题、判断题进行判题，排除证明题等verl数学验证库无法处理的题型。
2. 对齐Ground Truth格式，将数据集中的标准答案改写为符合SFT输出规范的格式（如去除 `\boxed{}`，统一使用 `$` 包裹等），确保比对基准的一致性。
3. 增强答案规范化（Normalization），在送入判题器之前，对模型输出和 Ground Truth 进行严格的清洗和归一化处理。具体的规范化策略包括：自动去除 LaTeX 中的冗余反斜杠（如 `\\\\` 转 `\\`）、修正特殊的空格转义（如 `\` 转空）、统一同义词（如将 对、True 统一为 `\checkmark`，错、False 统一为 `\times`），以及处理 `\mathbf{}` 等特殊字体格式。这种规范化大大提升了判题的鲁棒性，即使模型输出了细微的格式差异（如多写了一个反斜杠），也能被正确判定为正确。

通过这些改进，规则判题与人工判题的一致率从53%提升到了98.5%，为RL训练提供了更准确的奖励信号。

3.12 RL阶段训练数据格式

代码块

```
1  {
2    "conversations": [
3      {
4        "role": "system",
5        "content": "你是一个数学解题助手，能够帮助用户解决数学问题。"
6      },
7      {
8        "role": "user",
9        "content": [
10         {
11           "type": "text",
12           "text": "求解方程：2x + 3 = 7"
13         },
14         {
15           "type": "image_file_path",
16           "image_file_path": "/math/173a42bd2eb871a.png"
17         }
18       ]
19     }
20   ],
21   "question_id": "111",
22   "subject": "math",
```

```
23     "grade": 3,
24     "answer": "$x = 2$"
25 }
```

3.13 RL训练过程中，关键指标是如何变化的

- 1. 平均奖励：呈现震荡上升趋势，从约 0.32 持续波动上升至约 0.75。这直接反映了模型在训练过程中逐渐掌握了解题技巧，生成的答案越来越符合判题规则的要求，整体表现稳步提升。
- 2. KL散度：随着训练进行持续上升。这说明RL模型正在逐渐偏离原始的SFT基座模型，探索出与预训练阶段不同的解题策略。数值始终控制在较低范围（e-4量级），表明这种偏离是受控的，没有发生模型崩塌。
- 3. 策略熵：呈现先快速下降后缓慢回升的趋势。前期下降意味着模型迅速收敛，学会了固定的答题格式和套路；后期的缓慢回升表明模型在掌握基础后，开始尝试更多样化的推理路径，保持了一定的探索能力，防止陷入局部最优。
- 4. 平均回复长度：平均响应长度从 5500 左右逐渐下降并稳定在 3500-4000 之间。这表明模型在优化过程中学会了更加精简或符合特定长度约束的回复方式。
- 5. 梯度范数：梯度范数在 0.035 到 0.065 之间波动，整体未出现数值爆炸。这说明训练过程相对稳定，权重更新在可控范围内。
- 6. 回复长度裁剪比例：有多少比例的样本因为长度波动过大，触碰到了预设的裁剪阈值。从 0.28 迅速降至约 0.02。这体现了大模型的长度约束适应能力，在训练极早期就实现了回复长度的稳定。
- 7. 评测集指标：逐步提升到一个稳定的值，期间有一定波动但整体呈上升趋势。

3.14 具体指标

按图文类型分类的评测指标

评测集	初始值（纯文本方案）	基座模型（Qwen2.5-VL-32B）	CPT阶段（7B）	CPT阶段（32B）	SFT阶段（7B）	SFT阶段（32B）
清晰图评测集	58%	68%	73% (+5%)	73% (+5%)	79% (+6%)	84%
模糊图评测集	-	55%	60% (+5%)	60% (+5%)	71% (+11%)	72%
文本评测集	-	68%	71% (+3%)	71% (+3%)	74% (+3%)	78% (+7%)

按学龄分类的评测指标

评测集	基座模型 (Qwen2.5-VL-32B)	CPT阶段 (7B)	CPT阶段 (32B)	SFT阶段 (7B)	SFT阶段 (32B)	RL阶段 (7B)
小学评测集	70%	75% (+5%)	78% (+8%)	83% (+8%)	88% (+10%)	85% (+2%)
初中评测集	60%	65% (+5%)	65% (+5%)	71% (+6%)	75% (+10%)	73% (+3%)
高中评测集	55%	60% (+5%)	60% (+5%)	66% (+6%)	70% (+10%)	68% (+3%)

4. 八股问题

多模态相关

多模态SFT中，模型是如何处理多张图像输入的？

在处理多张图像输入时，本质上是将多组Visual Tokens嵌入到同一个文本序列中。

具体实现上，会在文本的Input IDs中，根据图片在对话中的实际位置，插入对应的Image占位符序列。比如第一张图的Token序列插入在第一轮对话前，第二张图插入在第二轮。当这些ID进入模型后，会被分别替换为对应图像的Visual Embedding。

这里有两个关键的技术细节。一是Position Embedding的处理，模型需要通过位置编码区分不同图像在序列中的先后顺序，确保模型知道哪张图对应哪个问题。二是Attention Mask的设计，在标准的Causal Mask基础上，需要确保文本Token能关注到其之前的所有图像Token，从而实现对多图上下文的联合理解。这样，模型就能处理像多题连问或者一道题包含多张步骤图这样的复杂场景。

多模态模型中，Vision Token和Text Token的Position Embedding是如何处理的？

Qwen2.5-VL采用了一种**顺延递增**的Position ID策略。模型不会为Vision Token和Text Token使用两套独立的位置编码，而是将Visual Tokens视为文本序列的一部分。

具体来说，当文本序列中遇到图片占位符时，Position ID会继续从上一个文本Token的ID开始递增，覆盖所有Visual Tokens的位置，然后后续的文本Token继续接在Visual Tokens的ID之后。例如，如果前缀文本占用了位置0-9，接着插入了256个Visual Tokens，那么这些Visual Tokens的Position ID就是10-265，后续文本从266开始。这种策略的优势是让模型能感知到模态间的**相对位置关系**。此外，配合RoPE，模型可以自然地处理任意长度的图文穿插序列，无需引入复杂的模态特定编码。

为什么 Visual Token 的数量通常是固定的（如 256/1024）？动态 Token 有什么优缺点？

在SFT训练中，我们实际上使用了**动态分辨率**，这意味着Visual Token的数量**不是固定的**，而是根据图片的分辨率和纵横比动态变化的。

- 固定Token数量（如CLIP的256）**的优点是计算方便，Batch对齐简单。但缺点是对于高分辨率的数学题大图，会强制压缩导致细节（如角标、细线）丢失；对于长条形的图表，会引入大量无效填充。

- **动态Token数量**：通过2D-RoPE机制，将图片切分为不同数量的Patch。优点是能保留原始分辨率的细节，对于数学公式识别至关重要。缺点是Batch训练时需要特殊的Padding或Packing策略来对齐不同长度的序列。

Qwen2.5-VL 的“原生动态分辨率”

- **核心逻辑**：Qwen2.5-VL 舍弃了将图像缩放或填充为固定正方形（如 224x224）的传统做法。它直接根据图像原始纵横比进行动态采样，将图像长宽调整为 28 的倍数，并通过 `grid_thw` 记录下 [T, H, W] 的结构信息。
- **工程优势**：这种方法不仅保留了图像的原始信息分布，更重要的是，它在表示空间特征时直接使用**实际坐标**，而非 0-1000 的归一化坐标。这避免了因长宽比缩放导致的物体几何形变，显著提升了模型对细长物体或复杂文档布局的理解能力。

MROPE 的与绝对时间对齐

- **MRoPE**：Qwen2.5-VL 将旋转位置编码（RoPE）从 1D 扩展为 3D，分别在时间（T）、高度（H）和宽度（W）三个独立维度上计算旋转量。
- **绝对时间对齐**：相比于 Qwen2-VL 简单的帧序号累加，Qwen2.5-VL 将时间维度的 ID 直接与物理时间戳对齐。这意味着在构建位置编码时，即便视频是以不均匀的 FPS（如跳帧采样）输入，模型也能通过时间 ID 之间的物理间隔感知真实的“时间流速”。这解决了模型对采样频率的敏感性，使其具备了秒级的精准时刻定位能力。

窗口注意力机制在 Vision Encoder 中的实现

- **设计初衷**：多模态模型在处理高分辨率或长视频时，视觉 Patch 数量会急剧增加，传统的全量自注意力计算复杂度为 $O(N^2)$ ，显存开销难以承受。
- **具体做法**：Qwen2.5-VL 在视觉编码器中引入了窗口注意力，将计算复杂度降为**线性增长**。模型并非全量使用窗口。架构中仅保留 4 层作为全局 Full Attention 以捕捉跨区域的长程依赖，其余层均使用 112×112 像素的局部窗口注意力。这种“全局+局部”的组合在推理效率和全局建模能力之间取得了帕累托最优。

为什么多模态模型普遍采用 ViT 而非CNN作为 Vision Encoder?

- **全局建模能力**：CNN 依赖感受野的逐层堆叠来捕捉全局信息，具有强烈的局部性偏置；而 ViT 核心的自注意力机制从第一层开始即可进行全局像素对齐，更适合处理多模态指令中复杂的长程关联。
- **归纳偏置的权衡**：CNN 的平移等变性和局部性在大规模预训练（Scaling Law）下不再是绝对优势。在大量数据预训练后，ViT 能够学习到比 CNN 更强的空间表征能力。

- **模态架构统一**：ViT 将图像序列化为 Token，与语言模型的 Transformer 结构天然兼容，便于进行联合训练和跨模态特征融合（如共享位置编码或注意力机制）。

Qwen2.5VL 的训练流程

1. 预训练阶段：

- **第一阶段：视觉编码器（ViT）训练**。在此阶段，模型主要利用海量的图文对（Image-Text Pairs）、OCR 数据和视觉知识数据，重点提升 ViT 提取高质量视觉特征的能力。
- **第二阶段：全参数联合训练**。激活 ViT 和 LLM 的全量参数，引入更复杂的交错图文数据（Interleaved Data）、多模态数学、视觉问答（VQA）以及纯文本数据。目标是建立视觉与语言模态间的深层逻辑联系，使模型具备初步的跨模态推理能力。
- **第三阶段：长上下文能力扩展**。专门针对长文本和长视频场景，将模型的序列处理长度从 8K 扩展到 32K。此阶段通过逐步增加训练窗口，确保模型在处理超长输入时位置编码（MROPE）的稳定性。

2. 后训练阶段：

- **SFT阶段**：训练数据包括图像-文本对、视频和纯文本数据，来自通用视觉问答、拒绝采样和专门数据集（如文档和OCR、定位、视频和代理任务）。
- **DPO阶段**：专注于图像-文本和纯文本数据，利用偏好数据来对齐模型与人类偏好。每个样本仅处理一次，以确保高效的优化。

说明 Vision Transformer (ViT) 的核心模块组成及其作用。

ViT 的架构设计遵循“将图像视为序列”的原则，主要包含以下四个核心模块：

- **Patch Embedding**：将输入图像切分为 N 个固定大小的不重叠块（patches）。每个块被展平并线性投影到维度 D，转化为模型可处理的 token 序列。在工程实现中，通常使用一个 `stride` 和 `kernel_size` 均为 patch 大小的 `Conv2d` 层来高效完成这一过程。
- **[class] token**：借鉴 BERT 的设计，在 patch embedding 序列前加入一个可学习的特殊 token。它旨在聚合整张图像的全局语义信息，其经过编码器处理后的输出向量被视为整张图的最终表示，用于下游分类任务。
- **Position Embedding**：由于 Transformer 的自注意力机制对输入顺序不敏感，ViT 为每个 token（包括 class token）添加了一个可学习的一维位置向量，以引入图像块之间的空间位置关系。
- **Transformer Encoder**：由多层相同的 Block 堆叠而成，每层包含多头自注意力（MSA）、前馈网络（MLP）、层归一化（LayerNorm）和残差连接。它通过全局注意力机制对 patch 之间的长距离依赖进行建模。
- **MLP Head**：在预训练阶段由带隐藏层的 MLP 构成，在微调阶段简化为单层线性映射，将 class token 的特征映射为类别概率。

为什么现在的多模态模型倾向于使用 MLP Projector 而不是 Q-Former?

1. Q-former使用少量可学习的Query Tokens，是一种有损的压缩，无论输入图片多大，输出到LLM的token数量都是固定的。对比MLP不会对视觉特征进行压缩，能最大限度地保留视觉特征的细粒度信息。
2. MLP架构更简单，更容易与LLM联合训练至收敛。而Q-Former 的模型架构更复杂且对超参数敏感，其Transformer结构使得训练更加复杂和不稳定。换句话说，在LLM能力足够强的今天，瓶颈往往不在对齐模块的复杂性，而在数据的质量。
3. Q-Former提取的语义往往是稀疏且重复的，体现在只能捕捉有限的视觉特征，且不同Query之间经常表达重复语义。这会导致在细粒度任务上表现更差，且更容易产生幻觉。

多模态预训练与指令微调阶段的优化目标有什么本质区别？

多模态训练通常分为两个阶段，它们承担的角色完全不同：

- **预训练：特征对齐**
 - **目标：** 让模型理解“这张图片里有个猫”和文本中“猫”这个词是等价的。
 - **策略：** 通常冻结 Vision Encoder 和 LLM，只训练 Projector。
 - **数据：** 使用大规模的 **Image-Caption 对**。此时模型在学习一种从视觉分布到文本空间的模态映射，而不是在学习对话。
- **指令微调：指令遵循**
 - **目标：** 让模型学会根据图像内容执行复杂指令（如：分析图片中的矛盾点、写代码生成图表）。
 - **策略：** 冻结 Vision Encoder，全量或使用 LoRA 微调 LLM，同时微调 Projector。
 - **数据：** 使用高质量的 **VQA 或对话数据**。这一阶段决定了模型的智商和对话风格。

强化相关

用通俗的语言解释GSPO的原理

想象你在教一个学生写作文：

- **GRPO的做法：** 我会逐字逐句地给每个词打分，这个词好给高分，那个词差给低分
- **问题：** 不同词的分数差别很大，有些词被大幅鼓励，有些词几乎没反馈，训练过程就像坐过山车一样不稳定

- **更严重的是：**在MoE模型中，同一个问题在不同训练阶段可能激活不同的"专家模块"，就像你今天用左手写字，明天突然换右手，这种不一致性会让模型彻底崩溃

GSPO核心理念：既然奖励是针对整篇文章的，那训练也应该针对整篇文章进行

GSPO的做法：

1. **整体评分：**不再逐词打分，而是给整个回答一个总体评价
2. **统一对待：**如果这个回答好，那么其中每个词都同等程度地被鼓励；如果回答差，每个词都同等程度地被惩罚
3. **长度归一化：**无论回答是10个字还是100个字，都用统一的标准来评判

GSPO对GRPO做了什么改进

- GRPO在token级别计算重要性权重，以修正新策略和旧策略之间的偏移，但这单个token对重要性可能波动很大，带来了高方差的训练噪声。这种噪声随着响应长度的增加而逐步累积，即随着链式乘积在序列层面不断增大。
- 并且被clip机制进一步放大，clip可能对有效梯度强行截断，破坏梯度方向一致性，等于在已高方差的权重上再做非线性“压扁/截断”。
- 这很容易导致模型崩溃，特别是MoE模型，因为在经过一轮或多轮梯度更新后，相同查询激活的专家可能会变化，导致 $w_{i,t}(\theta)$ 产生剧烈震荡。

GSPO的提出是为了解决 GRPO 在长序列和 MoE 模型训练中容易失稳的问题。GSPO 将优化粒度从 **token 级** 提升到 **序列级**，基于序列似然来定义重要性比率，并在序列层面执行奖励与优化。这样一来，训练信号更加一致，显著提升了训练的稳定性与效率，尤其在大规模 MoE 架构中优势突出。

奖励劫持

奖励劫持（Reward Hacking）是指在强化学习中，智能体利用奖励函数中的缺陷或模糊性来获取高奖励，而没有真正学习或完成预期的任务。奖励劫持的存在是因为强化学习环境通常是不完善的，并且准确地指定奖励函数从根本上具有挑战性。比如你想让机器人打扫房间，你得奖励设置为地面上垃圾越少，奖励越高，但是机器人发现，把所有垃圾都扫到地毯底下，或者用个大东西把垃圾盖住，传感器就检测不到垃圾了，于是能拿到超高分！但房间实际上还是脏的。

奖励劫持的产生主要有以下原因：

- **奖励函数设计缺陷：**奖励函数可能没有准确地反映设计者的意图，或者没有考虑到所有可能的情况，导致智能体可以找到捷径来获取高奖励。
- **环境复杂性：**强化学习环境可能非常复杂，智能体可能会发现一些设计者未预料到的行为来最大化奖励。
- **部分可观察状态：**智能体可能无法完全观察到环境的状态，从而导致其行为与设计者的预期不一致。

怎么缓解奖励劫持

1. 设计更智能、全面的奖励函数

- **多维度考量**：不要只盯着单一指标。例如，在打扫机器人场景下，除了统计“垃圾清理量”，还应对“将垃圾准确投掷进垃圾桶”这一关键动作进行奖励；
- **目标对齐**：奖励函数要尽量反映最终业务或任务目标，而非表面现象。这个过程需要不断试验与打磨；
- **引入惩罚项**：对一些明显偏离目标的行为（如机器人将垃圾藏匿）设置扣分或负奖励，增加防范强度。

2. 强化人工监督与定期审查

- **超越分数盲信**：奖励分数高并不意味着策略无懈可击，要定期检查 AI 实际执行的行为；
- **及时干预**：一旦发现模型开始钻空子，应迅速调整奖励函数，或通过人类反馈明确告知“不允许这样做”，类似 RLHF 中的人工反馈环节。

3. 多目标奖励与硬性约束

- **多信号协同**：在复杂任务中，单一奖励往往不够，可并行设置若干子目标，让模型在各个维度上都达到预期；
- **硬性规则**：对一些绝对不容违背的原则（如“禁止将垃圾扫入地毯下”）直接设为不可逾越的约束，而非靠奖励惩罚试图引导。

4. 正则化与行为规范性惩罚

- **KL 散度惩罚**：在 PPO 或其他策略优化中，常添加与原始“良好行为”模型（如 SFT 模型）之间的 KL 散度惩罚，以防新策略过度偏离、产生怪异行为；
- **限制偏移**：这样既能追求更高回报，又能控制模型行动风格，避免“剑走偏锋”。

5. 对抗性测试

- **模拟攻击者思维**：主动思考 AI 可能的取巧路径，设计专门的测试场景进行“红队”式攻击；
- **持续迭代**：针对发现的漏洞不断修补奖励函数或约束机制，提高系统鲁棒性。

6. 基于人类偏好的学习（RLHF）

- **学习偏好胜过完美奖励**：RLHF 不要求你精确定义所有奖励规则，而是通过人类偏好示例来训练奖励模型；
- **更接近直觉**：人类可以一眼分辨真正有价值的结果与“耍小聪明”的结果，这使得训练出的奖励信号更加健壮、贴近业务目标。

7. 简化原则

- **保持精简**：过于复杂的奖励机制更容易被模型钻漏洞；
- **直接明了**：在条件允许的情况下，尽量将目标与奖励设计得简洁、易于理解和验证。

VLLM相关

vllm是先进的大语言模型推理框架，核心技术是PagedAttention。PagedAttention通过将KV（Key-Value）缓存划分为固定大小的块（类似于虚拟内存中的页），实现了对内存的高效管理。这种设计允许系统根据实际需求动态分配和管理显存，从而提高了推理速度和资源利用率。

优化策略

- **连续批处理**：利用连续批次操作完成多个序列的推理，提高了GPU的利用率。
- **调度与抢占**：采用先来先服务的调度策略，并在需要时抢占资源，以处理更多的请求。
- **分布式管理**：优化了通信机制，提高了系统的扩展性和性能。

PagedAttention的优点

1. **页按需分配**：PagedAttention 将每个请求的 KV Cache 拆分为若干固定大小的页，并仅在生成新 tokens 时按需分配物理页，无需提前为最大序列长度预留内存，从而大幅降低 KV Cache 内存占用。
2. **跨请求 KV Cache 共享**：相同前缀的多个请求可共用 KV pages，大幅降低内存占用，适用于beam search、A/B test等场景。
3. **消除zero-padding**：传统方法往往需将所有序列填充到同一最大长度，导致短序列浪费大量内存和计算。PagedAttention 只在实际需要的 blocks 上分配内存，短序列不会因对齐而被额外填充，从而**zero-padding**问题被根本杜绝。
4. **提升推理吞吐**：由于减少了内存浪费和碎片，GPU 可以在同等显存下加载更多并发请求，vLLM 的实测表明 PagedAttention 能带来 2-4× 的整体吞吐提升，且在中长序列下优势更明显。

Continuous Batching

在大模型推理时，不再等一批请求都到齐才统一处理，而是把不同请求的 token 按时间轴拼接成一个“大批次”连续送入显卡，每个请求的状态（KV Cache）单独维护。

具体来说：

- 普通 batching 需要“齐头并进”，一批请求里最短的结束后，长请求还要继续跑，GPU 利用率会下降。
- Continuous batching 会把已经结束的请求踢出去，把新来的请求动态加入，把它们的输入 token 和历史 KV Cache 合并在一起，这样 GPU 始终保持高并发。
- 关键点是显存里要**独立保存每个请求的 KV Cache**，并用调度器管理不同请求的进度，才能保证模型对每个用户的上下文是独立正确的。

DP、TP、PP

- DP在多个GPU上复制模型，模型的参数存储在主GPU上，主GPU负责将模型参数广播到其他GPU上，多个GPU计算得到的梯度在主GPU上汇总，然后更新模型参数。
- PP：将模型的层切分到不同GPU，也叫层间并行或算子间并行（Inter-operator Parallel），不要求每张卡内都可以完整执行训练过程。面临流水线气泡问题（设备等待前向/反向传播的空闲时间）。
- TP：将单层内的张量操作（如矩阵乘法）拆分到多个设备，并行计算后合并结果，也叫层内并行或算子内并行（Intra-operator Parallel），不要求每张卡内都可以完整执行训练过程